Solution: Inserting a new node in a BST

Problem Statement

Write Recursive Approach to Insert New Node in a Binary Search Tree.

Given a binary search tree (BST) and a value to be inserted, write a recursive algorithm to insert a new node with the given value into the BST while maintaining its properties.

Examples

1. BST Before Insertion:

```
    4
   / \
  2   7
 / \
1   3
```

Input Node: 5 Output BST:

```
    4
   / \
  2   7
 / \   \
1   3   5
```

Explanation: The input node with value 5 is inserted as the right child of node 7.

2. BST Before Insertion:

```
    6
   / \
  3   8
 / \   \
1   5   9
```

Input Node: 4 Output BST:

```
    6
   / \
  3   8
 / \   \
1   5   9
   /
  4
```

Explanation: The input node with value 4 is inserted as the left child of node 5.

3. BST Before Insertion: Empty BST (null) Input Node: 2 Output BST:

```
  2
```

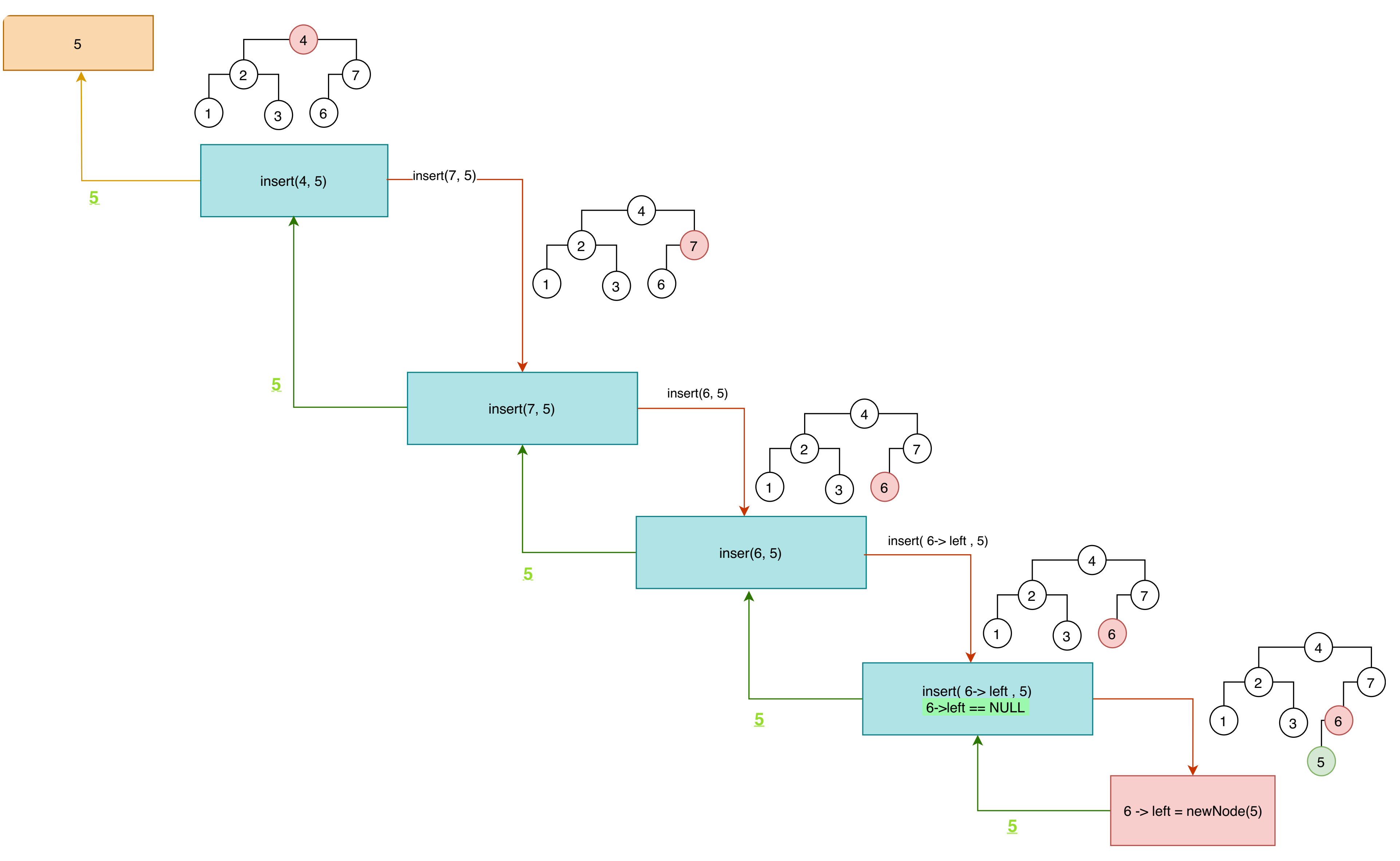
Explanation: The input node with value 2 becomes the root of the BST.

Solution

The recursive approach to insert a new node in a BST can be summarized as follows:

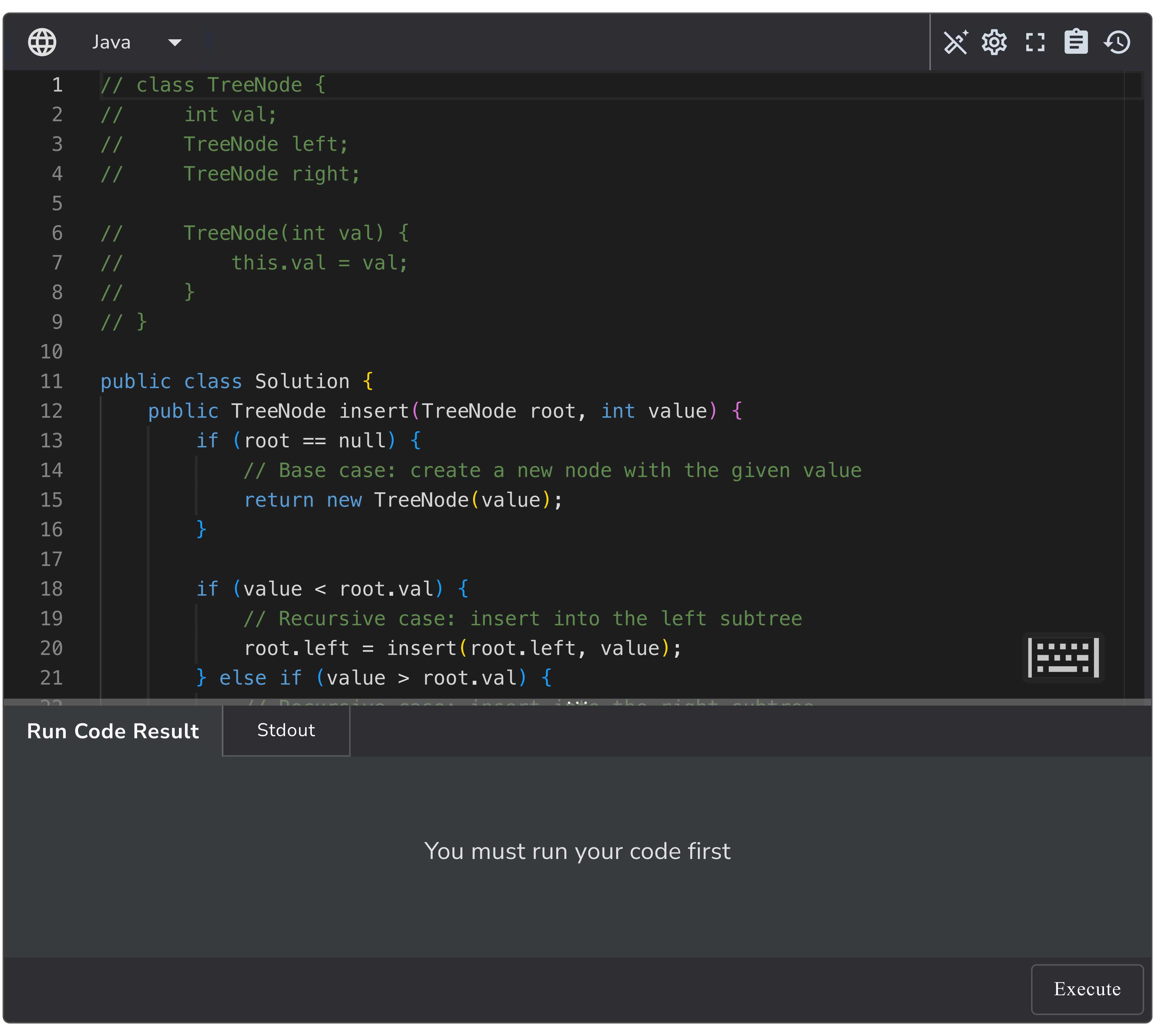
- If the BST is empty (null), create a new node with the given value and return it.
- If the value to be inserted is less than the value of the current node, recursively call the insert function on the left subtree.
- If the value to be inserted is greater than the value of the current node, recursively call the insert function on the right subtree.
- Finally, return the modified BST.

This approach works because it follows the property of a BST, where all nodes in the left subtree are smaller than the current node, and all nodes in the right subtree are greater than the current node.



Code

Here is the code for this algorithm:



```java
// class TreeNode {
//     int val;
//     TreeNode left;
//     TreeNode right;

//     TreeNode(int val) {
//         this.val = val;
//     }
// }

public class Solution {
    public TreeNode insert(TreeNode root, int value) {
        if (root == null) {
            // Base case: create a new node with the given value
            return new TreeNode(value);
        }

        if (value < root.val) {
            // Recursive case: insert into the left subtree
            root.left = insert(root.left, value);
        } else if (value > root.val) {
```

You must run your code first

Time and Space Complexity

The time and space complexity of the algorithm is $O(h)$, where h is the height of the BST. In the worst case, where the BST is skewed (i.e., a linked list), the height h is equal to the number of nodes in the BST, resulting in $O(N)$ time and space complexity. However, in a balanced BST, the height h is logarithmic to the number of nodes, resulting in efficient operations with $O(logN)$ time and space complexity.

← Previous
10. Inserting A New Node In A BST

Next →
11. BST Inorder Traversal

☐ Mark as Completed